

SQL Query Reference

Everyday SQL patterns: filtering, joins, aggregation, window functions and tuning.

Logical execution order

Column aliases from SELECT are therefore usable in ORDER BY but not in WHERE.

```
FROM → JOIN → WHERE → GROUP BY → HAVING
      → SELECT → DISTINCT → ORDER BY → LIMIT
```

Joins

Join	Returns
INNER JOIN	Rows with a match on both sides
LEFT JOIN	All left rows; NULLs where the right side has no match
RIGHT JOIN	All right rows; NULLs where the left side has no match
FULL OUTER JOIN	All rows from both sides
CROSS JOIN	Cartesian product of both inputs

Aggregation

- COUNT(*) counts rows; COUNT(col) skips NULLs.
- String aggregation: GROUP_CONCAT (MySQL), STRING_AGG (Postgres/SQL Server).
- Filter inside an aggregate with FILTER (WHERE ...) in Postgres.

```
SELECT category, COUNT(*) AS n, AVG(size) AS avg_size
FROM files
WHERE deleted_at IS NULL
GROUP BY category
HAVING COUNT(*) > 2
ORDER BY n DESC;
```

Window functions

- Window functions keep every row while aggregates collapse rows.

```
ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY created_at DESC)
RANK()        OVER (PARTITION BY category ORDER BY downloads DESC)
SUM(size)     OVER (ORDER BY created_at)           -- running total
LAG(price)    OVER (PARTITION BY sku ORDER BY day) -- previous value
```

Subqueries & CTEs

- EXISTS short-circuits as soon as one row matches — usually faster than IN for large sets.
- CTEs read better than nested subqueries; Postgres 12+ inlines them when possible.

```
WITH recent AS (  
    SELECT * FROM files WHERE created_at > NOW() - INTERVAL '30 days'  
)  
SELECT category, COUNT(*) FROM recent GROUP BY category;
```

Indexes & tuning

- B-tree indexes serve equality, range and ORDER BY — the default choice.
- A composite index (a, b) helps WHERE a = ? AND b = ? but not WHERE b = ? alone.
- Always index foreign-key columns used in joins; check with EXPLAIN ANALYZE.
- Functions on indexed columns (WHERE LOWER(email) = ...) disable the index unless a functional index exists.
- Covering indexes let a query be answered from the index alone (index-only scan).

Safety habits

- Always write the WHERE clause before the SET clause in UPDATE/DELETE.
- SELECT first with the exact WHERE clause, then convert to UPDATE/DELETE.
- Wrap multi-statement changes in BEGIN ... COMMIT and verify with a SELECT before COMMIT.
- Parameterise all user input to avoid SQL injection.